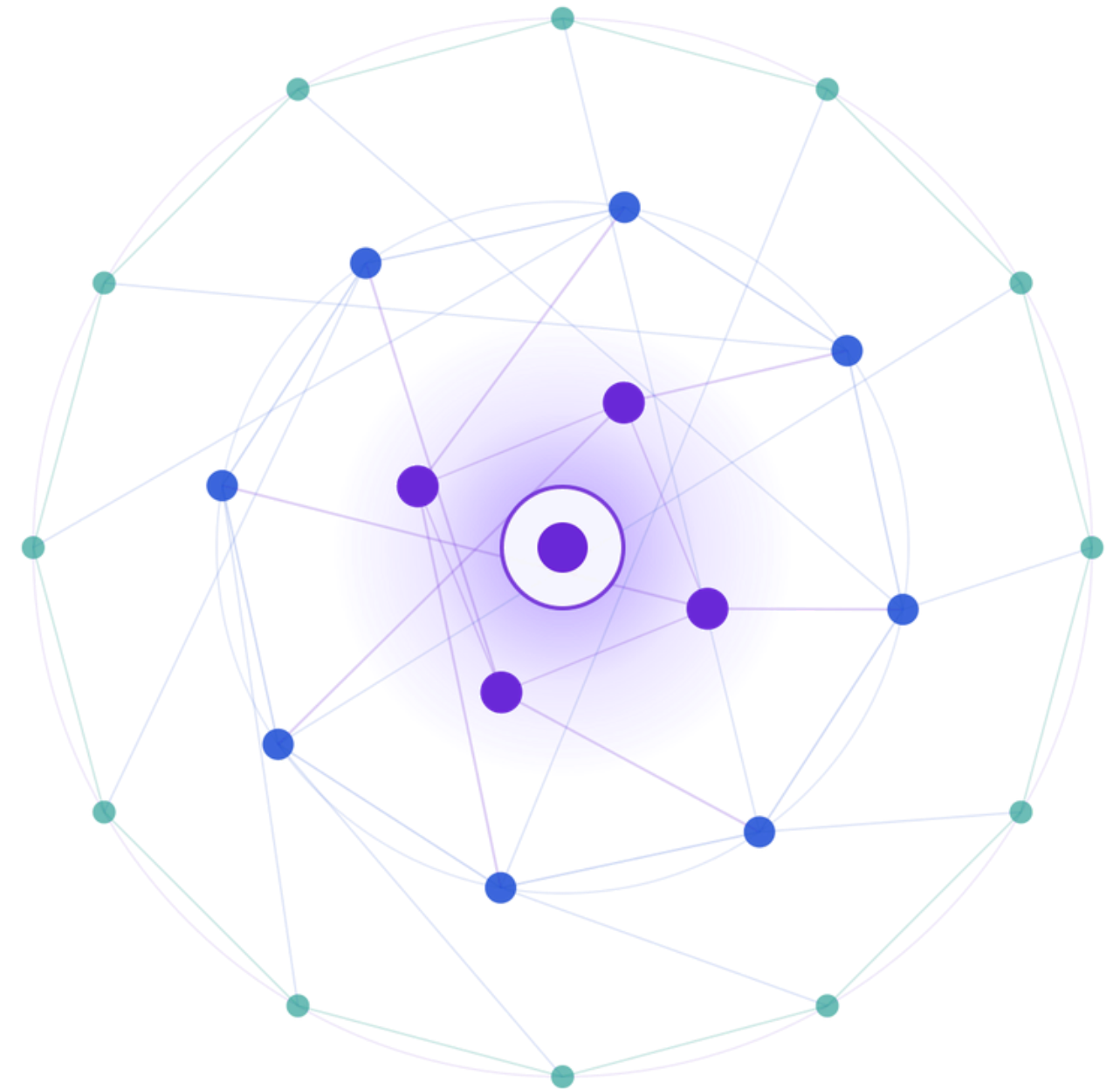


Generative AI Hackathon

3rd Edition 2026 -27

- Theme ID - Theme 2 (SmartGuided TroubleshootingEngine)
- Team Name - Code & Chill_SRM
- College Name - SRM UNIVERSITY OF SCIENCE AND TECHNOLOGY
- Team Lead Name & Email 1 - Asleshita Tuduku (asleshita.t@gmail.com)
- Member Name & Email 2 - Dhruthi (dhruti.p.2007@gmail.com)
- Member Name & Email 3 - Bhavya Sri (bsri110725@gmail.com)
- Submission Github link - <https://github.com/Asleshita/prism-troubleshoot>



THEME

- Theme 2 - Smart Guided Troubleshooting Engine
- Problem: Samsung support agents manually translate raw internal knowledge-store (SIIS) articles into step-by-step guidance for customers - slow and inconsistent.
- Goal: build POST /v1/troubleshoot - an API that takes a user's query plus a raw SIIS article and returns a structured, deeplink-enriched troubleshooting guide (Goals -> Actions -> StepGroups), automatically.

Existing Solutions & Gaps

- Existing Samsung support flow: customer describes issue -> agent manually reads SIIIS article -> agent manually decides which Settings screens to point to -> inconsistent phrasing and missed deeplinks.
- Gap 1: No automatic structuring - raw SIIIS text is unstructured prose, not actionable steps.
- Gap 2: No deeplink discovery - agents don't reliably know which of the 578 Settings deeplinks applies to a given step.
- Gap 3: No caching - the same or rephrased question is answered from scratch every time.
- Gap 4: No safety net - a flaky LLM call means no answer at all for the customer.

Our Solutions & Architecture Diagram

- Our solution: a pipeline that turns (query, SIIS article) into a validated Context Deeplink Response in one API call.
- Architecture:
 - query + siis_response -> cache check (by article content hash)
 - cache miss -> Gemini drafts goal / title / steps / description / category from the SIIS text only
 - Gemini failure or timeout -> automatic rule-based fallback derives steps directly from the SIIS text, so the API never errors or returns empty
 - deeplink matcher attaches real Settings deeplinks by matching each step's own wording against the 578-entry catalog - never invents a URI
 - format repair + validation layer enforces the goal regex, 2-3 word title, "It will" + 5-7 word description, score range, and strips any URL before returning

Demo & Product Walkthrough

- Live demo: GET /health returns {"status": "ok"} - confirms the API is up.
- POST /v1/troubleshoot with a real query (e.g. "My Galaxy S22 screen is completely black and won't turn on") and its SLS article returns a full ContextDeeplinkResponse with Goals, Actions, StepGroups and real deeplinks.
- Demo video: [YOUR VIDEO LINK HERE]

Tools and tech stack used

- Python 3.11, FastAPI, Pydantic v2 (schema validation against the official schema.py)
- Google Gemini (gemini-3.1-flash-lite) for query understanding and step drafting
- In-memory cache keyed on SIIIS article content hash, for sub-300ms repeat/paraphrase responses
- Rule-based fallback generator (no external dependency) as a reliability safety net
- Docker for reproducible deployment

Impact & Use case

- Turns every raw Samsung support article into a ready-to-use, structured troubleshooting flow - no manual agent authoring needed.
- Customers get consistent, jargon-free steps with one-tap Settings deeplinks instead of generic text instructions.
- Cuts response time for common issues (blank screen, touch problems, screen mirroring, etc.) from manual lookup to a sub-second cached API call.
- Scales directly to any new SIIS article without retraining or rewriting rules, since steps are always derived live from the source text.

Innovation highlights, results and limitations

- Innovation: deeplinks are matched by meaning (validation key / description), never invented - the LLM never writes a deeplink URI itself.
- Innovation: automatic fallback generator guarantees a valid, non-empty response even if the LLM is down, slow, or rate-limited.
- Results: 20/20 kit scenarios pass every format rule (goal regex, title length, description rule, score range, auto-actions-have-deeplinks).
- Results: zero URL leaks across all responses; cold-start latency under the 8-second limit.
- Limitation: deeplink matching favors precision over recall - it may correctly leave a step as "manual" rather than risk attaching a wrong deeplink.
- Limitation: fallback phrasing is plainer than the LLM path, since it exists purely as a safety net.

What's next

- Move deeplink matching from keyword/label matching to embedding-based semantic matching for higher recall.
- Add a persistent cache (Redis) so warm results survive a server restart.
- Expand generalization testing against a wider set of unseen SIIS articles and query phrasings.
- Add automated regression tests that run `scripts/selfcheck.py` on every commit.

Brownie points slide(differentiation)

- Full safety net: every response is guaranteed schema-valid even under LLM failure, thanks to the rule-based fallback.
- Zero-URL-leak enforcement is applied as a final regex sweep over the entire JSON output, not just individual fields.
- results.jsonl includes 8-10 genuinely diverse query_variations per query (synonym swaps, register changes, reordering, question framing) - not near-duplicates.
- Self-check script (scripts/selfcheck.py) reproduces the judges' own checks locally, including cold/warm latency per query.

Checklist- Updated on Public GitHub

- Working prototype code - public GitHub repo: Y
(<https://github.com/Asleshita/prism-troubleshoot>)
- README with reproducible setup instructions: Y
- Demo video, max 5 minutes: [YOUR VIDEO LINK HERE]
- Presentation file (PPT or PDF): Y

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India